

รายงาน

เรื่อง Project Proposal

เสนอ

ดร. บุญหทัย เครือแก้ว

จัดทำโดย

67050437 ภาณุวัฒน์ รอดตัวดี

67050473 รัชชานนท์ เหล่าสัมพันธ์

รายงานนี้เป็นส่วนหนึ่งของรายวิชา 05506258 CLOUD COMPUTING AND VIRTUALIZATION TECHNOLOGY

1. Project Overview

1.1 ชื่อโปรเจกต์

Live Auction Platform

ระบบประมูลสินค้าออนไลน์แบบเรียลไทม์ที่ผู้เข้าร่วมประมูลหลายคนสามารถเห็นราคาล่าสุดอัพเดททันทีพร้อมกัน

1.2 ที่มาและปัญหา (Pain Point)

ระบบประมูลออนไลน์ทั่วไปที่ใช้สถาปัตยกรรมแบบ request-response (HTTP polling)
มีข้อจำกัดสำคัญคือผู้ใช้ต้องกดรีเฟรชหรือระบบต้อง poll ข้อมูลซ้ำ ๆ เพื่อเช็คว่าราคาปัจจุบันเปลี่ยนไปหรือยัง
ซึ่งทำให้เกิดความหน่วง (latency) และประสบการณ์การประมูลไม่สมจริง นอกจากนี้เมื่อมีผู้เข้าร่วมประมูลหลายคนกด "bid"
ในราคาเดียวกันพร้อมกันในเสี้ยววินาที ระบบที่ออกแบบไม่ดีจะเกิด race condition — ปัญหาที่พบได้ในระบบจอง
ระบบสต๊อกสินค้า และระบบจองที่นั่ง

กลุ่มผู้ใช้เป้าหมาย: กลุ่มวัยรุ่นหรือ Community ขนาดเล็กที่ต้องการจัดประมูลของใช้/ของสะสมที่ไม่เป็นทางการ เช่น ประมูลการ์ด เกม เป็นต้น

1.3 Project Goal & Scope

เป้าหมายหลัก: สร้างระบบประมูลเรียลไทม์ที่รับประกันความถูกต้องของราคา โดยใช้สถาปัตยกรรม serverless เต็มรูปแบบ ไม่มี server ที่ต้องรันตลอดเวลา

In-Scope:

- ระบบสมัครสมาชิก/เข้าสู่ระบบพื้นฐาน
- สร้างรายการประมูล (สินค้า, ราคาเริ่มต้น, ราคาขั้นต่ำที่เพิ่มได้ต่อครั้ง, เวลาสิ้นสุด)
- เข้าร่วมประมูลแบบเรียลไทม์ผ่าน WebSocket ให้เห็นราคาล่าสุดของทุกคนพร้อมกันทันที
- ระบบป้องกัน race condition ด้วย DynamoDB conditional write
- ระบบปิดประมูลอัตโนมัติเมื่อหมดเวลา พร้อมประกาศผู้ชนะประมูล
- ประวัติการ bid ของแต่ละรายการประมูล

Out-of-Scope:

- ระบบชำระเงินจริง (payment gateway)
- ระบบจัดส่งสินค้า
- ระบบยืนยันตัวตนขั้นสูง (KYC)

2. สถาปัตยกรรมของแอปพลิเคชัน (Application Architecture)

2.1 Tech Stack

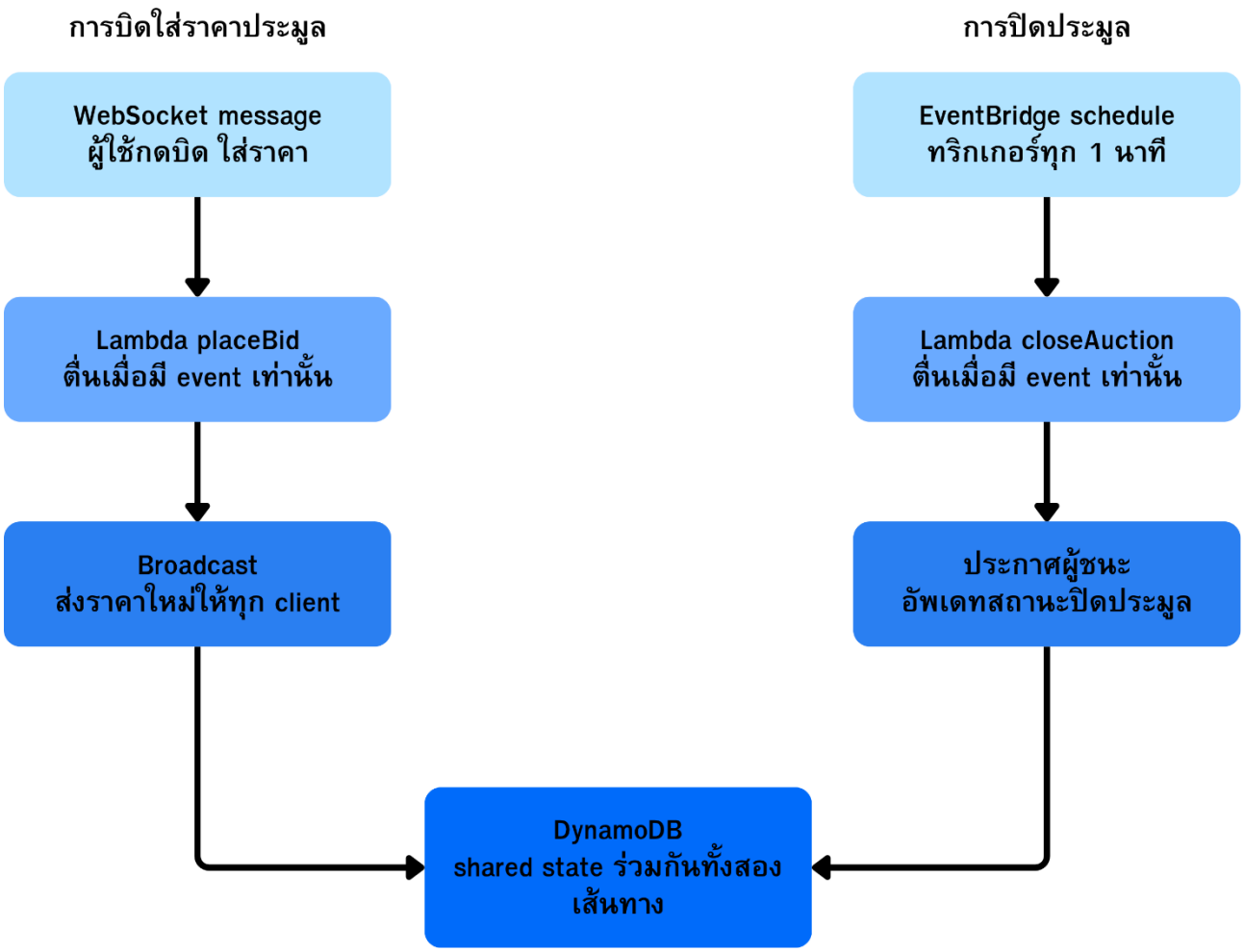
Component	Technology	Reasons
Frontend	React	Component-based, จัดการ WebSocket connection state ได้สะดวกด้วย hooks
Backend	Node.js (AWS Lambda functions)	Cold start เร็วกว่า Python/Java บน Lambda เหมาะกับ WebSocket event handler ที่ต้องตอบสนองไว
Real-time Layer	Amazon API Gateway (WebSocket API)	จัดการ persistent connection แบบ serverless ไม่ต้องมี server รันค้างตลอดเวลา
Database	Amazon DynamoDB	รองรับ conditional write แบบ atomic ซึ่งเป็นหัวใจของการป้องกัน race condition free tier ให้ throughput เพียงพอสำหรับ scope โปรเจกต์
Scheduled Task	Amazon EventBridge	ตรวจสอบและปิดประมูลที่หมดเวลาอัตโนมัติ

2.2 รูปแบบสถาปัตยกรรม (Pattern)

ระบบใช้สถาปัตยกรรมแบบ Serverless Event-Driven ทั้งหมด ไม่มี server ที่ต้องดูแลการรันตลอดเวลา ทุกส่วนของโปรแกรมจะคิดเงินตามการใช้งานจริง (pay-per-use)

ระบบแยกความรับผิดชอบเป็น 3 Lambda function ได้แก่ $connect (เปิดการเชื่อมต่อ), placeBid (ประมวลผล business logic), และ $disconnect (ปิดการเชื่อมต่อ) ทำให้แต่ละส่วนทดสอบและ scale ได้อิสระจากกัน

2.3 Diagram: การเชื่อมต่อระหว่างส่วนต่างๆ



3. Cloud Infrastructure Architecture

3.1 Cloud Provider

เลือก Amazon Web Services (AWS) ด้วยเหตุผลดังนี้:

- Free Tier ครอบคลุมบริการหลักที่ต้องใช้ทั้งหมด (Lambda, API Gateway, DynamoDB, EventBridge)
- API Gateway รองรับ WebSocket API แบบ native
- DynamoDB มี conditional write และ atomic operation ที่จำเป็นสำหรับการแก้ปัญหา concurrency
- เอกสารและตัวอย่างการใช้งานสำหรับ serverless WebSocket pattern มีครบถ้วน เหมาะกับทีมที่กำลังเรียนรู้ pattern นี้เป็นครั้งแรก

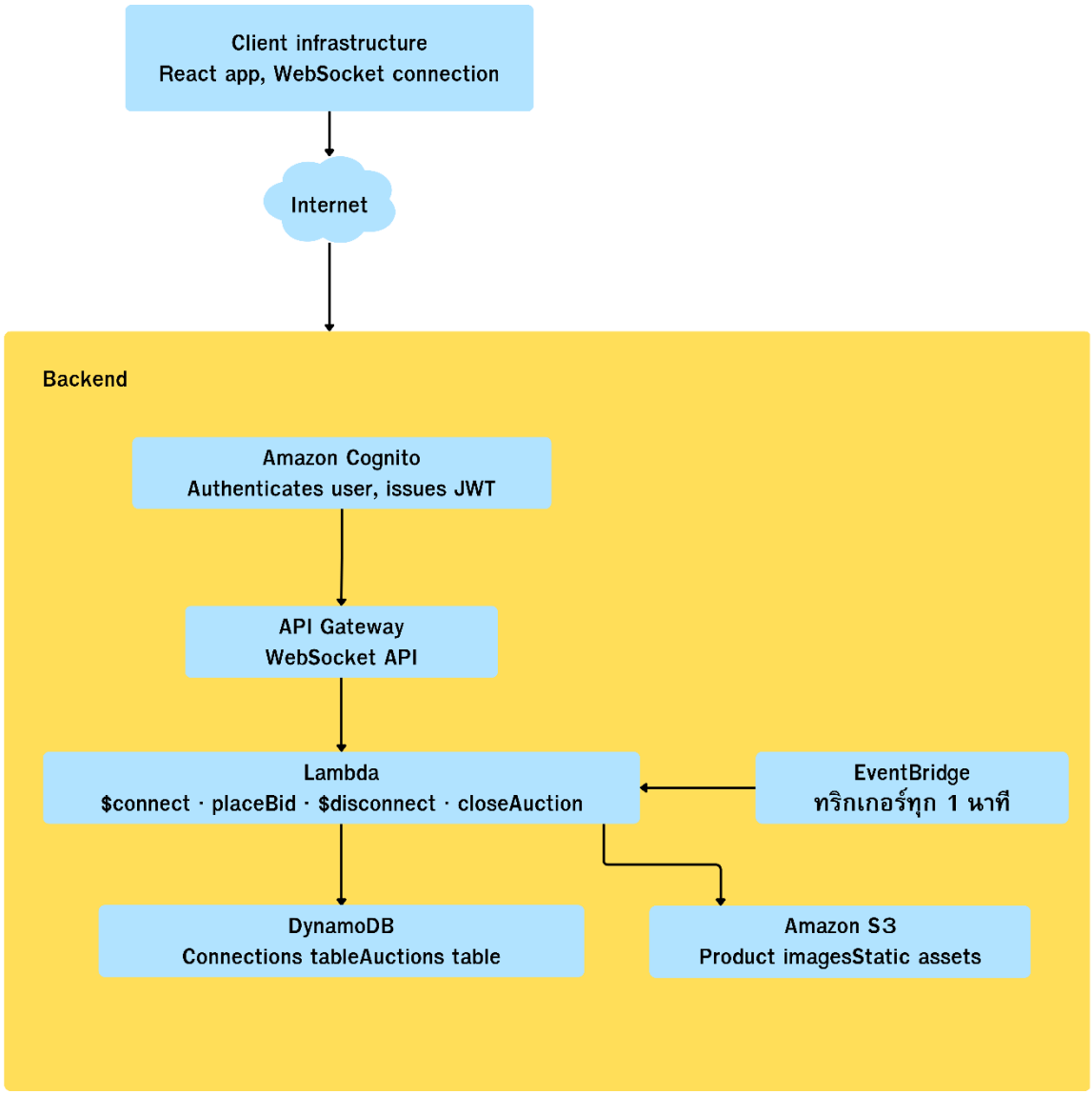
3.2 Service Mapping

Category	Services	Reason
Compute	AWS Lambda	ไม่ต้องดูแล server, จ่ายตามการใช้งานจริง
Real-time Messaging	API Gateway (WebSocket API)	จัดการ connection ของผู้ใช้ให้เอง ไม่ต้องเขียน WebSocket server เอง แถมส่งข้อมูลกลับไปหาทุกคนพร้อมกันได้ในตัว
Database	Amazon DynamoDB	DynamoDB มีฟีเจอร์ที่ "เขียนข้อมูลได้ก็ต่อเมื่อข้อมูลนั้นยังไม่มีใครแตะ" ทำให้เวลาคนบิดราคาพร้อมกันหลายคน ราคาจะไม่มีวันเพี้ยน
Scheduled Jobs	Amazon EventBridge	คอยเช็คทุก 1 นาทีว่ามี การประมูลไหนหมดเวลาหรือยัง เพราะระบบจะรู้เรื่อง การบิดราคาก็ต่อเมื่อมีคนกดบิดเท่านั้น ถ้าไม่มีใครบิดตอนหมดเวลาพอดี ก็ต้องมีตัวคอยเช็คว่าจบการประมูลแล้ว
Storage	Amazon S3	เก็บรูปภาพสินค้าที่นำมาประมูล และ static asset ของ frontend

Networking

เนื่องจากสถาปัตยกรรมเป็น serverless ทั้งหมด จึงไม่จำเป็นต้องออกแบบ VPC/Subnet แบบดั้งเดิม Lambda functions เชื่อมต่อกับ DynamoDB ผ่าน AWS internal network โดยตรง ลดความซับซ้อนด้าน network configuration และลดความเสี่ยงด้าน security ที่มักเกิดจากการตั้งค่า Security Group/Subnet ผิดพลาด

3.4 แผนภาพ Cloud Architecture



3.4 Data Model (DynamoDB)

Table	Partition Key	Detail
Connections	connectionId	เก็บคู่ connectionId กับ auctionId เพื่อใช้ broadcast ลบทิ้งเมื่อ disconnect
Auctions	auctionId	เก็บราคาปัจจุบัน, ผู้ชนะล่าสุด, เวลาสิ้นสุด เขียนผ่าน conditional write เท่านั้น

ความเสี่ยงทางเทคนิคที่ต้องวางแผนล่วงหน้า: WebSocket API คิดค่าใช้จ่ายตาม connection-minute และจำนวน message ที่ส่ง แม้จะอยู่ใน Free Tier แต่ทีมควรทดสอบ load เบื้องต้นก่อนวันนำเสนอ เพื่อประเมิน quota ที่ใช้จริงเมื่อมีผู้ทดสอบพร้อมกันหลายคน

4. กลยุทธ์การ Deploy และ Automation (CI/CD Strategy)

ทีมวางแผนใช้ GitHub Actions เป็น CI/CD pipeline หลัก ในการเชื่อมต่อกับ repository บน GitHub โดยมีขั้นตอนดังนี้:

1. Continuous Integration: ทุกครั้งที่ push โค้ดเข้า branch หลัก ระบบรัน automated test (unit test สำหรับ Lambda function logic โดยเฉพาะ conditional write logic) และ lint check
2. Continuous Deployment: เมื่อ test ผ่านทั้งหมด ระบบ deploy Lambda functions และ infrastructure ผ่าน AWS SAM (Serverless Application Model) หรือ Serverless Framework โดยอัตโนมัติ
3. Environment Separation: แยก stage dev และ prod เพื่อทดสอบก่อน deploy จริง ป้องกันการทดลองกระทบข้อมูลที่ใช้นำเสนอ

การใช้ Infrastructure as Code (SAM/Serverless Framework) แทนการตั้งค่าผ่าน AWS Console ด้วยมือ ช่วยให้ทีมสามารถสร้าง environment ใหม่ได้ซ้ำ ๆ อย่างสม่ำเสมอ และง่ายต่อการ rollback หากเกิดปัญหา

5. แผนการจัดการความปลอดภัยและค่าใช้จ่าย

5.1 Security

- Authentication: ใช้ Amazon Cognito สำหรับจัดการผู้ใช้และออก JWT token ทุก WebSocket message ต้องแนบ token เพื่อยืนยันตัวตนก่อนดำเนินการ bid
- Authorization: ตรวจสอบสิทธิ์ระดับ Lambda function ก่อนอนุญาตให้แก้ไขข้อมูล auction
- Data Integrity: DynamoDB conditional write ป้องกันทั้ง race condition และการเขียนข้อมูลที่ไม่สอดคล้องกัน
- Idempotency: ออกแบบ Lambda function ให้ปลอดภัยหาก AWS retry การเรียกซ้ำ (เช่น เครือข่ายกระตุก) โดยไม่ทำให้ bid ถูกนับซ้ำ

5.2 Cost Management

Services	Free Tier Limit	Control Plan
AWS Lambda	1M requests/เดือน, 400,000 GB-seconds compute	ตั้งค่า memory allocation ให้เหมาะสมกับงานแต่ละ function ไม่ over-provision
API Gateway WebSocket	1M messages, 750,000 connection-minutes (12 เดือนแรก)	จำกัดจำนวนผู้ทดสอบพร้อมกันช่วง demo ปิด connection ที่ idle นานเกินไปอัตโนมัติ
DynamoDB	25GB storage, 25 WCU/RCU (on-demand ตามจริง)	ใช้ on-demand capacity mode แทน provisioned เพื่อไม่เสียค่าใช้จ่ายช่วงไม่มีการใช้งาน
EventBridge	14M events/เดือน	ตั้ง schedule rule ให้ทำงานทุก 1 นาที เพียงพอต่อ scope โปรเจกต์ ไม่ถี่เกินจำเป็น

จะมีตั้ง AWS Budget Alert แจ้งเตือนเมื่อค่าใช้จ่ายประมาณการเกิน $1 เพื่อป้องกันการใช้งานเกิน Free Tier โดยไม่ตั้งใจ

6. แผนการดำเนินงาน (Timeline & Milestones)

ช่วงเวลา	Milestone	รายละเอียด
25 – 29 สิงหาคม 2569	Setup & Design	ตั้งค่า AWS account, ออกแบบ DynamoDB schema, สร้าง repository และ CI/CD pipeline เบื้องต้น
30 สิงหาคม – 9 กันยายน 2569	Core Backend	พัฒนา Lambda functions ($connect, $disconnect, placeBid) พร้อม conditional write logic และ unit test
10 – 12 กันยายน 2569	Real-time Integration	เชื่อมต่อ API Gateway WebSocket กับ frontend, ทดสอบ broadcast แบบหลาย connection พร้อมกัน
13 – 17 กันยายน 2569	Auto-close & Auth	เพิ่มระบบปิดประมูลอัตโนมัติผ่าน EventBridge, integrate Cognito authentication
18 – 22 กันยายน 2569	Frontend Polish	ปรับปรุง UI/UX, responsive design, แสดงประวัติการ bid
23 – 30 กันยายน 2569	Testing & Load Test	ทดสอบ concurrency จริง (หลายคน bid พร้อมกัน), ตรวจสอบ cost ใน AWS Billing Dashboard
1 – 3 ตุลาคม 2569	Documentation & Presentation	จัดทำเอกสารประกอบ, เตรียม slide และซ้อมนำเสนอ